User Guide for Visual Studio's Refactoring Tools

Refactoring is the process of altering the code such that it is easier to understand, maintain, and extend, all without changing any existing behavior. Visual Studio is natively equipped with the following refactoring commands:

Command	Description
Rename	Renames identifiers for class names, fields, local variables, types, method names, and all other code symbols.
Extract Method	Moves a selection of code into its own method. Replaces the original selection of code with a call to this new method.
Change Signature	Reorders or removes parameters for a method. Updates all calls to this method with the new signature.
Introduce Constant	Replaces a "magic" string or number with a constant.
Introduce Local	Converts inline expressions into a local variable.
Inline Temporary Variable	Converts a local variable into inline expressions.
Encapsulate Field	Creates a new property that matches an existing field, setting that existing field to private in the process.
Extract Interface	Transfers a member originating from an existing struct, class, or interface, into a newly generated interface.

Accessing the Refactoring Tools

These refactoring commands are available in Visual Studio's code editor. To access these tools:

1. Highlight the section of code you wish to refactor.
2. Right click the selected code, and choose "Quick Actions and Refactoring…" The keyboard shortcut "Ctrl+." will also accomplish this.

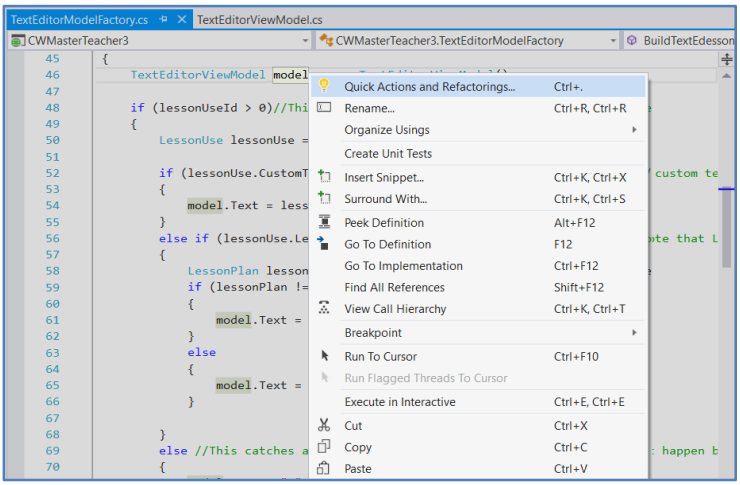

Figure 1 – Accessing the Quick Actions Menu

After completing the two steps listed above, the Quick Actions menu (which can be identified with a lightbulb icon, as seen in *Figure 2* below) will appear near the selected code. The options available in this menu are a subset of Visual Studio's refactoring commands, with each option being context sensitive to the highlighted section of the code.

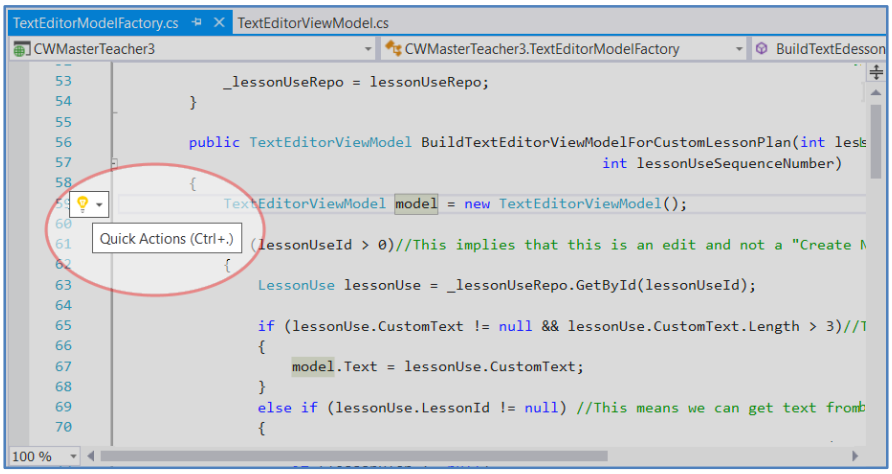

Figure 2 – The Quick Actions Drop Down Menu

Using Quick Actions and Preview Changes

Once Quick Actions has opened, you can invoke a refactoring command by selecting the desired command's option from the drop down menu. This will bring up that command's dialog (further details on each of these dialogs can be found in the *Using the Refactoring* Tools section).

Dependent on the refactoring command selected, a "Preview changes" option may be available. When this option is selected, Visual Studio will open a preview dialog before applying its changes to the code, as can be seen in *Figure 3*. This dialog box is split into two sections; the upper section contains a tree of the proposed changes, where branches represent where the change will occur on a file level, and leaves represent where the change will occur actually within the file. Each change is checked by default but may be unchecked to stop that change from being made. The lower section of the dialog box contains a snippet of code that previews the changes that will be made. Clicking the button labeled "Apply" will commit those changes to the code.

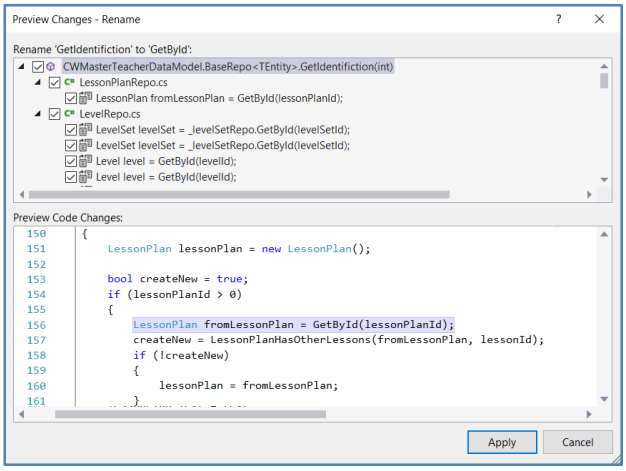

Figure 3 - The Preview Changes dialog box

Using the Refactoring Tools

After selecting a refactoring command from the Quick Actions menu, a dialog box specific to that command will appear. This section outlines how to operate each of those commands.

- ## Rename

  This command allows you to change the name of an item and apply that change to all locations that reference the renamed item. Rename is available from a number of places within Visual Studio. It may be accessed within the code editor by:
  - placing the cursor on the element to be renamed, then using the keyboard shortcut "Ctrl+R, Ctrl+R",
  - right clicking an element to be renamed, and selecting "Rename…",
  - typing over the old name to make the Quick Actions (light bulb icon) menu appear. From there, select the rename option.

  The editor then highlights the item being renamed while this operation is active. It includes options to preview changes, and options to apply the rename to comments and strings as well.

- ## Extract Method

  The "Extract Method" command takes a selection of code and extracts it to a new method, retaining all existing behavior. The extracted code gets replaced with a call to this new method. To apply this tool, select the portion of code that you wish to extract, access the Quick Actions menu and select "Extract Method". There is an option to preview the changes made by this operation before making them. The method will be added below the enclosure where it was the original code was extracted from. After the extraction, Visual Studio will automatically invoke the "Rename" command on the newly created method.

- ## Change Signature (Reorder Parameters/Remove Parameters)

  This refactoring command allows a method or constructor's signature to be altered either by removing or by reordering parameters. The "Change Signature" command

  can be accessed through the Quick Actions menu after highlighting a method, indexer, or delegate. Selecting the "Change Signature" option from the Quick Actions menu brings up a dialog box that lists each parameter, and by default, highlights the first parameter. Clicking the arrow buttons that appear on this dialog box will reposition the highlighted parameter, while clicking the "Remove" button will remove the highlighted parameter. This refactoring tool incudes the option to preview any changes before making them. Once the signature change is applied, all calls to this method are replaced with its new signature.

- ## Introduce Constant

  The "Introduce Constant" tools allows magic numbers and strings to be pushed into constant variables. It is accessed through the Quick Actions menu, after selecting a magic value. You will have to select whether a constant should be introduced only in the highlighted location, or if all occurrences of the value should be changed. After this decision is made, there are several options available. "Introduce Constant For" creates a standard constant scoped at the class

level. “Introduce Local Constant For” creates a constant with a local scope. Choosing to introduce a constant for all occurrences will go beyond the magic value’s scope and find all other uses of that value. This refactoring command includes the ability to preview changes, along with the option to check for the selected magic value within comments and strings.

- ## Introduce Local

  This command allows an expression of code to be converted to a local variable. It is accessed through the Quick Actions menu, after selecting an expression. You will have to select whether a local should be introduced only in the highlighted location, or if all occurrences of the expression need to be changed. It also includes the ability to preview changes, along with the option to check for the selected magic value within comments and strings.

- ## Inline Temporary Variable

  The “Inline Temporary Variable” command replaces all references to a temporary variable with the code expressions assigned to that temporary variable. This is essentially the inverse operation of the “Introduce Local” command. It is accessed through the Quick Actions menu, after selecting a temporary variable. This refactoring command also includes a preview changes option.

- ## Encapsulate Field

  This refactoring command easily creates properties out of existing fields. It may be accessed through the Quick Actions menu, after highlighting a field. Within the Quick Actions menu, an additional option exists for the “Encapsulate Field” operation, labeled with “usages reference field”. This option converts all of the chosen field’s existing references into a reference of the newly created property from

  that field. When the encapsulation is applied, the existing internal field is set to be private (if it wasn’t already). If the field is defined as read-only, the property will include a *get* accessor for the field; otherwise, both *get* and *set* methods will be included.

- ## Extract Interface

  The “Extract Interface” command allows you to use existing class, struct, or interface to automatically create a new, matching interface. This tool may be accessed through the Quick Actions menu, after selecting the existing target class, struct, or interface to extract from. After selecting this option from the Quick Actions menu, the “Extract Interface” dialog box will appear, containing a field to define the name for the new interface, along with a checklist holding the possible public members that can be used to form the new interface. Unchecking any of these members will exclude them from the generated interface. Clicking the “OK” button creates the interface, and makes the existing class, struct, or interface implement this newly created interface.